

GE 120 1A

Introductory Object-Oriented Programming for Geomatics

Final Project

Submitted by:

Group 5

DE JESUS, Mary Catherine

TORRECAMPO, Charles Allen

MAÑAGO, Rizh Keilah

Submitted to:

Edgardo Macatulad

May 21, 2026

I. Introduction

Leveling is a surveying method used to measure the elevation of different points on the ground. In geomatics and construction, this process is necessary for mapping out land, planning infrastructure, and is the foundation for almost any site project.

While collecting the data in the field is straightforward, processing it back in the office can be a problem. Doing the math for 3-wire or differential leveling manually involves a lot of repetitive calculations, which takes a lot of time. More importantly, doing it manually makes it very easy to make blunders, which makes the whole survey unusable.

To solve this, our group built a Python program to automate the work. The program reads the raw leveling data directly from a CSV file. It then calculates the elevations, the misclosure error, and applies adjustments to find the corrected elevations. In the end, it generates a report of the computations and final values in either a text file or an HTML file, along with a graph of the elevation profile.

II. Objectives

Develop a Python program that processes surveying data and calculates point elevations. The program must do the following:

1. Read the Input Data

- Open and read leveling data from CSV files (leveling_data.csv and leveling3wire_data.csv).
- Successfully handle both **differential leveling** and **3-wire leveling** data formats.

2. Perform the Calculations

- Calculate the elevation of each survey point.
- Calculate the total error (**misclosure**) and determine the **order of accuracy** achieved.
- Apply formulas to distribute the error and calculate the final, **adjusted elevations**.

3. Generate the Report

- Export the final results into a cleanly formatted table.

- Include a summary of the misclosure and accuracy level.
- Save the output as either a **text (.txt)** or **HTML (.html)** file.

III. General Description of Program

Program Overview

The Leveling Computation Tool is a desktop/web application that automates the computational workflow of differential and three-wire spirit leveling surveys. Given a CSV data file and a benchmark elevation, the program computes instrument heights, station elevations, total distance, error of misclosure, order of accuracy, and distance-weighted adjusted elevations and presents all results together with an elevation profile plot and exportable reports.

I. Supported leveling types

- Differential Leveling: data columns: BS_point, BS_Reading, BS_Dist, FS_point, FS_Readings, FS_Dist
- Three-Wire Leveling: data columns: BS_point, BS_upper, BS_middle, BS_lower, FS_point, FS_upper, FS_middle, FS_lower

The tool exists in two implementations that share identical computation logic: a Jupyter Notebook prototype (object-oriented Python) and a Streamlit web application (function-based, browser-hosted). This report describes both in unified terms.

II. Materials & Platform

The table shows all the materials and the platform used in order to achieve the objectives. Its role and type are also indicated for a clearer understanding of how it was utilized during the programming.

Table 1. Materials and Platform used

Category	Item	Role	Type
----------	------	------	------

Language	Python 3.x	Core computation, data manipulation, file I/O	Runtime
UI Framework	Streamlit	Browser-based interface, widgets, state management	Framework
Data	pandas	CSV parsing, DataFrame operations, column arithmetic	Library
Visualisation	Matplotlib	Elevation profile chart generation and PNG export	Library
Math	math (stdlib)	Square root for allowable misclosure thresholds	Stdlib
I/O	io (stdlib)	In-memory buffers for TXT, HTML, and PNG download	Stdlib
Prototype Env.	Jupyter Notebook	Interactive OOP prototype; step-by-step development	Platform
Input Format	CSV file	Raw field data; two accepted schemas	Data

III. Methodology

A. Data Loading & Type Detection

The program reads a user-supplied CSV into a pandas DataFrame and inspects its column names against two known schemas. If all three-wire columns are present, the survey is classified as 3-Wire; if all differential columns are present, it is Differential. Any other schema raises a format error.

B. Wire Averaging (3-Wire only)

For three-wire surveys, each row's upper, middle, and lower stadia readings are averaged to produce a single backsight reading (BS_Reading) and foresight reading (FS_Readings), each rounded to 3 decimal places. Stadia distances are computed as $(\text{upper} - \text{lower}) \times 100$ for both backsight and foresight.

C. Height of Instrument & Elevation

Starting from the benchmark elevation (BM), the program iterates row by row:

1. $HI = \text{current elevation} + BS_Reading$: Height of Instrument at each setup.
2. $Elevation = HI - FS_Readings$: Unadjusted elevation of the foresight station.
3. The newly computed elevation becomes the current elevation for the next row.

D. Misclosure & Order of Accuracy

Error of misclosure is the difference between the sum of all backsights and the sum of all foresights. The total circuit distance D (in kilometres) drives the allowable tolerances per FGCS standards.

Table 2. Misclosure and its Corresponding Order of Accuracy

Misclosure	Order of Accuracy
$ e \leq 0.004\sqrt{D}$	First Order
$ e \leq 0.008\sqrt{D}$	Second Order
$ e \leq 0.0012\sqrt{D}$	Third Order
$ e \geq 0.004\sqrt{D}$	Rejected

E. Elevation Adjustment

Adjusted elevations distribute the misclosure proportionally by cumulative distance from the benchmark:

$$Adj_Elev = Elevation + (-misclosure / total_dist) \times Dist_from_BM$$

F. Report Generation

The tool renders the results table, a matplotlib elevation profile, and provides three downloadable export formats: plain-text report (TXT), HTML table report, and profile chart (PNG).

IV. Flowcharts

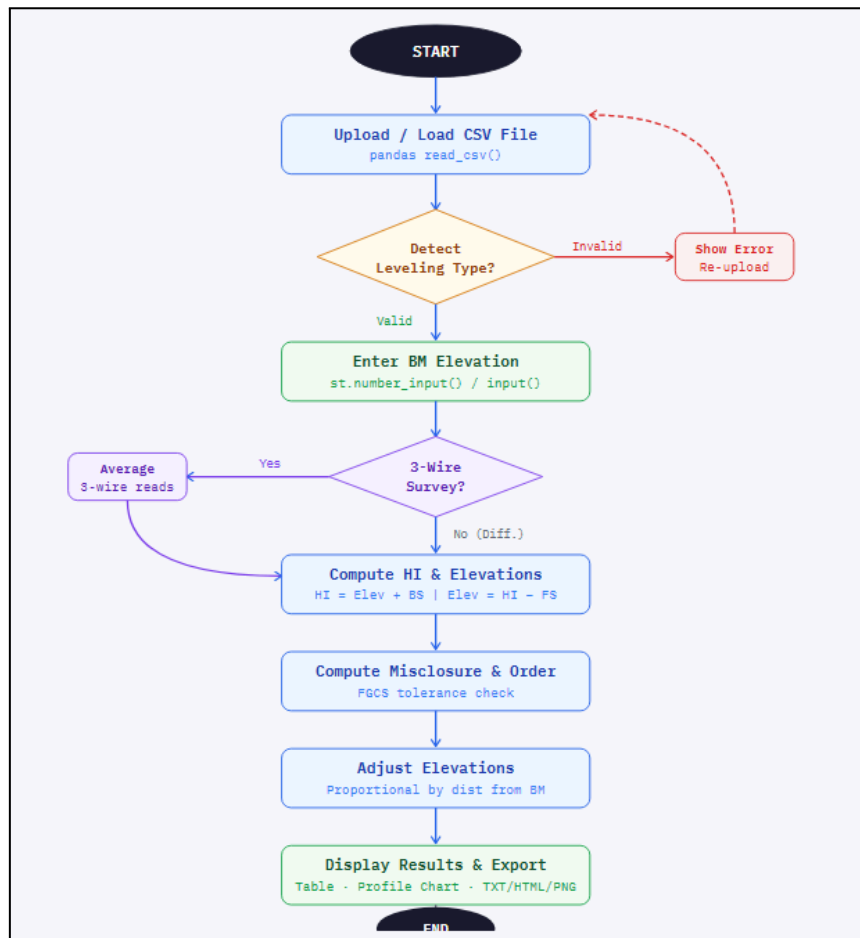


Figure 1. Overall Program Flow

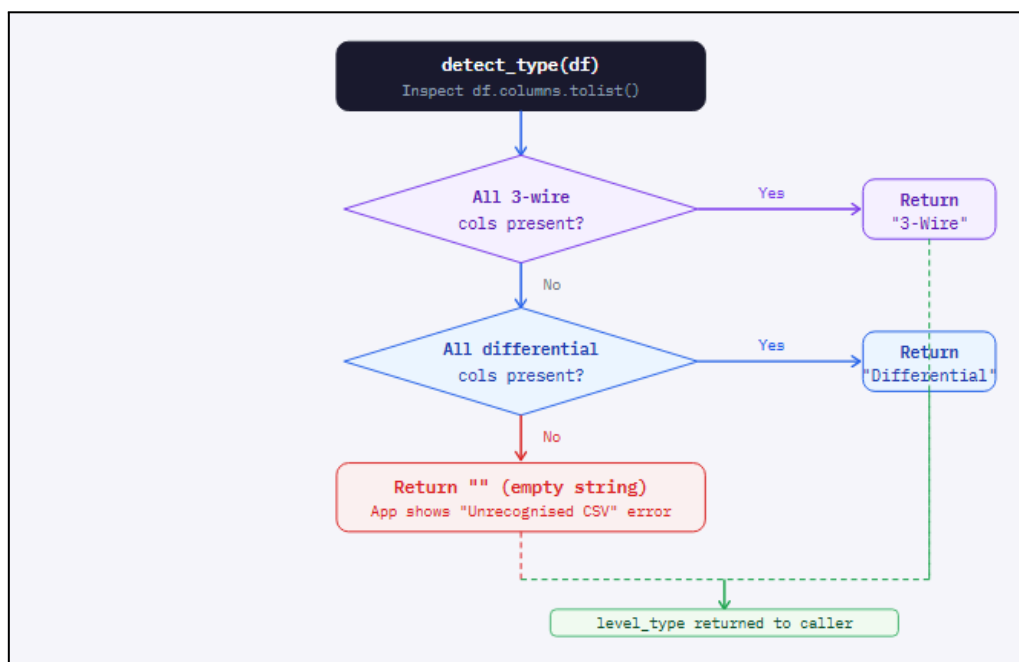


Figure 2. CSV Detection & Validation Logic

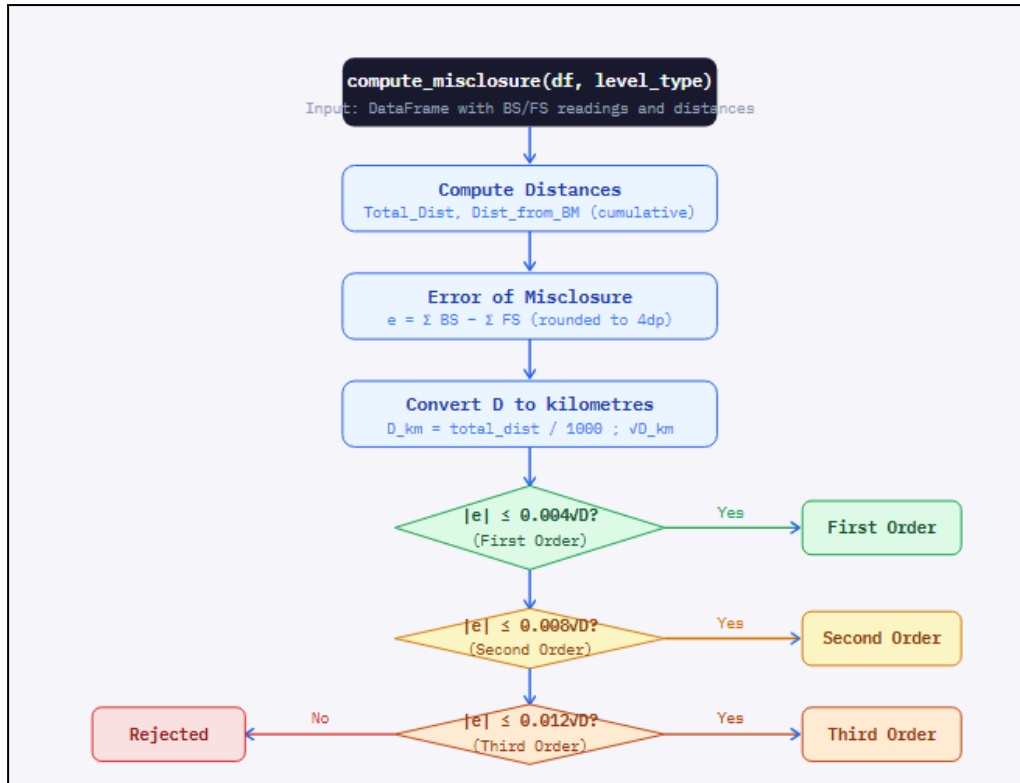


Figure 3. Misclosure & Accuracy Classification

UML Class Diagram of Notebook Prototype

The Jupyter Notebook prototype uses three collaborating classes. The Streamlit app ports the same logic into stateless functions but retains the same conceptual boundaries shown below.

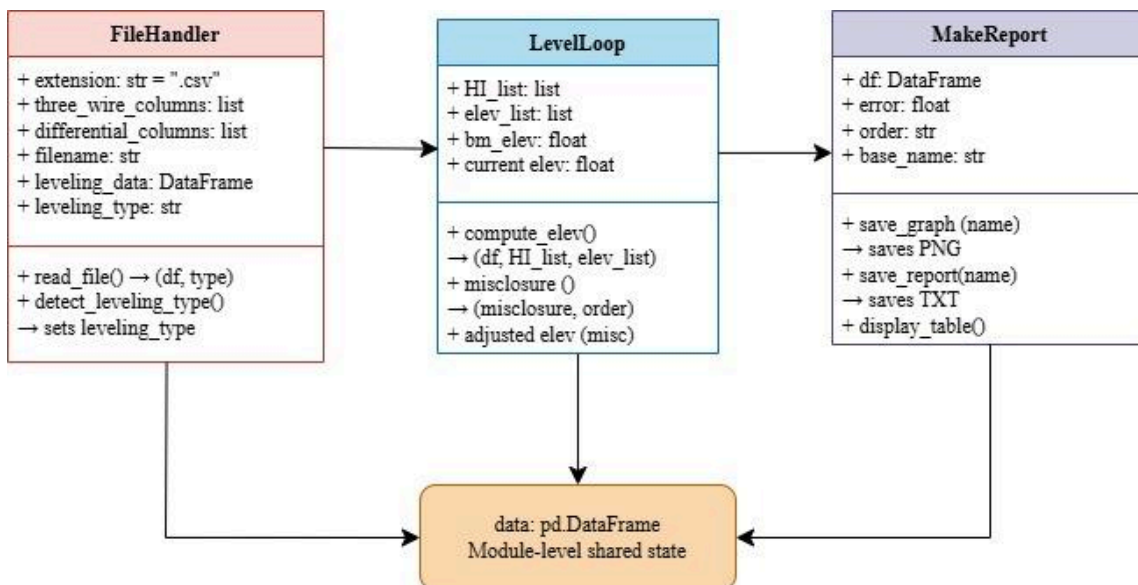


Figure 4. UML Diagram

Streamlit App Architecture

The leveling_app.py Streamlit application refactors the notebook classes into five pure functions and a session-state layer, enabling a reactive browser UI with sidebar controls and a main results area.

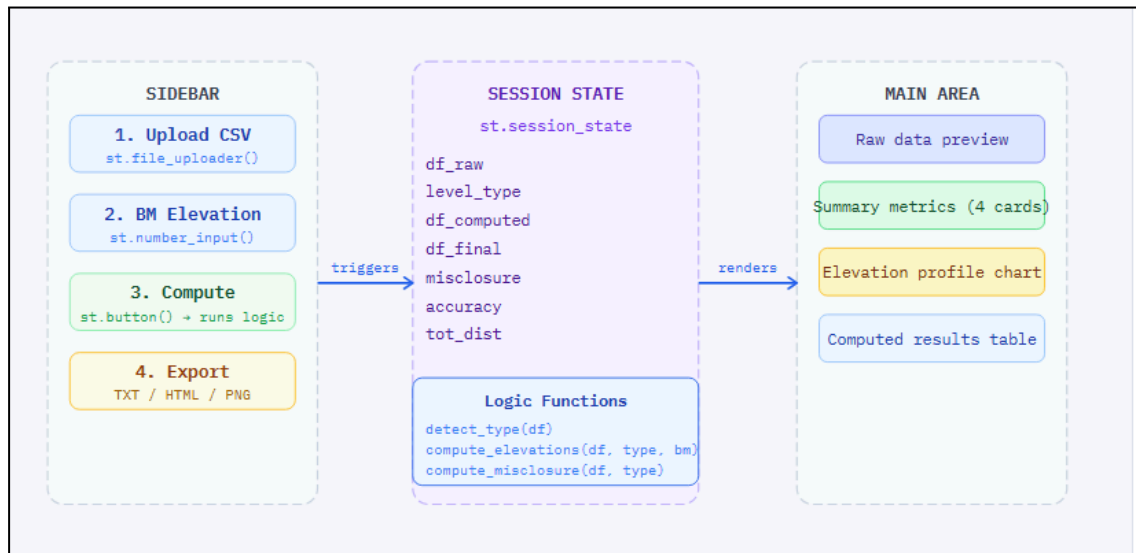


Figure 5. Streamlit Flowchart

IV. Results and Discussion

The developed program successfully automated the processing of differential and three-wire leveling data. The program was able to read both leveling_data.csv and leveling3wire_data.csv, detect the appropriate leveling type, and perform the required computations for elevations, misclosure, order of accuracy, and adjusted elevations.

For the outputs, the program generated complete computation tables together with elevation profile graphs in PNG format. It also successfully exported the results into TXT and HTML reports for easier viewing and documentation. The generated outputs were organized and consistent with standard leveling procedures used in surveying.

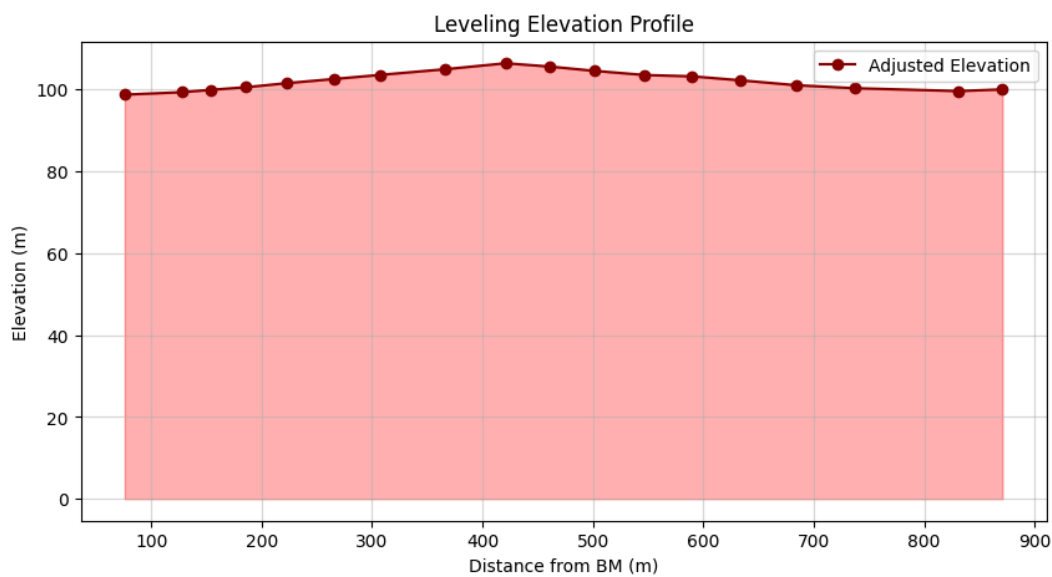


Figure 6. Sample Generated Leveling Profile (PNG Format)

```

--- LEVELING COMPUTATION REPORT ---

```

BS_point	BS_Reading	BS_Dist	FS_point	FS_Readings	FS_Dist	Elevation	Total_Dist	Dist_from_BM	Adj_Elev
BM1	0.700	38	TP 1	1.990	38	98.710	76.0	76.0	98.717417
TP 1	1.530	26	TP 2	0.930	26	99.310	52.0	128.0	99.322491
TP 2	1.570	15	TP 3	1.000	11	99.880	26.0	154.0	99.895029
TP 3	1.660	16	TP 4	1.020	16	100.520	32.0	186.0	100.538152
TP 4	1.820	18	TP 5	0.836	19	101.504	37.0	223.0	101.525762
TP 5	1.750	20	TP 6	0.726	23	102.528	43.0	266.0	102.553959
TP 6	1.790	22	TP 7	0.790	20	103.528	42.0	308.0	103.558057
TP 7	1.910	30	TP 8	0.550	28	104.888	58.0	366.0	104.923718
TP 8	2.210	28	TP 9	0.756	27	106.342	55.0	421.0	106.383085
TP 9	1.010	18	TP 10	1.820	22	105.532	40.0	461.0	105.576989
TP 10	0.610	18	TP 11	1.650	22	104.492	40.0	501.0	104.540892
TP 11	0.700	22	TP 12	1.730	24	103.462	46.0	547.0	103.515381
TP 12	0.710	22	TP 13	1.040	21	103.132	43.0	590.0	103.189577
TP 13	0.800	22	TP 14	1.780	22	102.152	44.0	634.0	102.213871
TP 14	0.690	26	TP 15	1.880	24	100.962	50.0	684.0	101.028751
TP 15	0.913	27	TP 16	1.650	26	100.225	53.0	737.0	100.296923
TP 16	1.020	52	TP 17	1.730	42	99.515	94.0	831.0	99.596096
TP 17	2.510	22	BM 1	2.110	18	99.915	40.0	871.0	100.000000

```

--- MISCLOSURE AND ACCURACY SUMMARY ---
Error of Misclosure: -0.085
Order of Accuracy: Rejected / Low Accuracy

```

Figure 7. Sample Generated Leveling Report (TXT Format)

Leveling Computation Report

BS_point	BS_Reading	BS_Oht	FS_point	FS_Reading	FS_Oht	Distance	TSB_Oht	CH1_Inst_BH	Adj_BH
BH1	0.000	36	TP 1	1.000	36	96.710	16.0	16.0	96.711417
TP 1	1.000	26	TP 2	0.000	26	96.710	62.0	128.0	96.702641
TP 2	1.000	16	TP 3	1.000	11	96.000	26.0	154.0	96.696029
TP 3	1.000	16	TP 4	1.000	16	100.000	32.0	186.0	100.000162
TP 4	1.000	16	TP 5	0.000	16	101.004	31.0	223.0	101.007462
TP 5	1.000	20	TP 6	0.000	23	102.000	43.0	266.0	102.003659
TP 6	1.000	22	TP 7	0.000	20	102.000	43.0	309.0	102.009877
TP 7	1.000	30	TP 8	0.000	28	104.000	50.0	360.0	104.002716
TP 8	2.200	28	TP 9	0.000	27	106.242	55.0	415.0	106.200005
TP 9	1.000	16	TP 10	1.000	22	106.000	40.0	455.0	106.000000
TP 10	0.000	16	TP 11	1.000	22	104.462	40.0	505.0	104.540002
TP 11	0.000	22	TP 12	1.000	24	103.462	40.0	545.0	103.610001
TP 12	0.000	22	TP 13	1.000	21	103.132	43.0	588.0	103.690077
TP 13	0.000	22	TP 14	1.000	22	102.162	44.0	632.0	102.740011
TP 14	0.000	26	TP 15	1.000	24	100.002	60.0	692.0	101.000161
TP 15	0.013	27	TP 16	1.000	26	100.225	53.0	735.0	100.266023
TP 16	1.000	52	TP 17	1.000	42	96.015	94.0	829.0	96.090006
TP 17	2.000	22	BH 1	2.000	18	96.015	40.0	871.0	100.000000

Misclosure and Accuracy Summary
Error of Misclosure: -0.005
Order of Accuracy: Reduced/Low Accuracy

Figure 8. Sample Generated Leveling Report (HTML Format)

The use of automation significantly reduced repetitive manual calculations and minimized possible arithmetic errors during computation. Compared to manual processing, the program provided faster and more systematic results while following the correct leveling workflow. However, the program still relies on properly formatted CSV files and accurate field observations to produce reliable outputs.

Overall, the program achieved the expected objectives and demonstrated how Python can be applied in automating common geomatics and surveying computations.

V. Conclusion

The leveling computation tool was able to meet the main objectives of the project. Specifically, it can read CSV files, detect the type of leveling data, compute elevations, determine the error of misclosure, classify the order of accuracy, apply proper elevation correction, and generate exportable reports with an elevation profile graph.

This project shows how Python can be used to automate common surveying computations in geomatics. Additionally, the use of object-oriented programming made the program more organized, reusable, and easier to understand. The use of pandas, matplotlib, and file-handling methods also made the workflow more efficient.

In conclusion, the program provides a faster and more systematic way of processing differential and three-wire leveling data. It helps minimize manual calculation errors and produces clear outputs that can be used for checking, documentation, and reporting. While the program can still be improved with more validation features and better handling of field blunders, it already serves as a practical tool for basic leveling computation and report generation.

VI. Members' Contribution

Member	Code Contribution	Paper/Report Contribution
DE JESUS, Mary Catherine	● Implemented class revisions (Objective 2). ● Coded misclosure, accuracy, and accumulative distance calculations (Objective 3). ● Contributed to applying the adjusted elevation formula (Objective 4). ● Coded the GUI with streamlit and matplotlib libraries	● General Description of Program
TORRECAMPO, Charles Allen	● Developed initial formulas (Objective 2). ● Coded the report outputs for leveling computation, misclosure, and accuracy (Objectives 5 & 6).	● Introduction ● Objectives ● Members' Contribution ● Appendix
MAÑAGO, Rizh Keilah	● Coded data reading from CSV (Objective 1). ● Implemented class revisions (Objective 2). ● Coded misclosure calculations (Objective 3). ● Contributed to applying the adjusted elevation formula (Objective 4).	● Results and Discussion (R&D) ● Conclusion

VII. References

[1] Python Software Foundation, "math — Mathematical functions," *Python 3.x Documentation*. [Online]. Available: <https://docs.python.org/3/library/math.html>.

[2] Pandas Development Team, "pandas User Guide," *pandas Documentation*. [Online]. Available: https://pandas.pydata.org/docs/user_guide/index.html.

[3] Pandas Development Team, "API Reference," *pandas Documentation*. [Online]. Available: <https://pandas.pydata.org/docs/reference/index.html>.

[4] Matplotlib Development Team, "Matplotlib Users Guide," *Matplotlib Documentation*. [Online]. Available: <https://matplotlib.org/stable/users/index.html>.

[5] Streamlit Inc., "API Reference," *Streamlit Documentation*. [Online]. Available: <https://docs.streamlit.io/develop/api-reference>.

VIII. Appendix

Repository Link: <https://github.com/catorrecampo-sys/-GE-120-1A-Final-Project-Group-5>